

GUÍA DEL ASISTENTE DE INVESTIGACIÓN

Sistema

BCI-SSVEP asíncrono para telepresencia robótica

Control de un TurtleBot3 mediante señales cerebrales

FASE 1

Documento: Guía del Asistente — Fase 1
Versión: 2.0
Basada en: Bitácora Técnica 3C2 **v3.1**
Código: Seis paquetes Python (*_F1)
Entregables: 8
Requisitos: Python 3.8+, ROS2 Humble, PsychoPy

QUÉ ES ESTE DOCUMENTO

Contiene todo lo necesario para trabajar de forma autónoma en el proyecto: qué problema se resuelve y por qué, cómo funciona el sistema, el manual completo del código, y ocho entregables con instrucciones precisas.

No hay calendario. El trabajo avanza por entregables, no por fechas. Cada sesión de revisión se dedica a discutir lo completado desde la anterior. El ritmo lo marca el avance real.

SOBRE LA FASE 1

El proyecto se desarrolla en dos fases. Esta guía y el código que la acompaña corresponden a la **Fase 1**.

La **Fase 2** es una extensión posterior que aún no está implementada. Lo que sí está preparado es su punto de conexión: hay un bloque de código delimitado y desactivado, listo para que quien la desarrolle lo active. Se explica en la Parte 3.

No necesitas saber nada más sobre la Fase 2 para completar los ocho entregables.

ANTES DE EMPEZAR: LEE LA SECCIÓN DE PROTOCOLO DE ERRORES

El código que recibes es un **borrador funcional**. Se ha probado y las 279 pruebas automáticas pasan, pero **no ha sido verificado en el hardware del laboratorio**.

Es tu responsabilidad leerlo, entenderlo, y corregir los errores que encuentres. Cuando corrijas algo, documenta qué estaba mal, por qué, y cómo lo arreglaste.

Varios de los errores que ya se corrigieron durante su desarrollo **no producían ningún mensaje de error**: el programa corría y devolvía números perfectamente creíbles que no significaban nada.

Contents

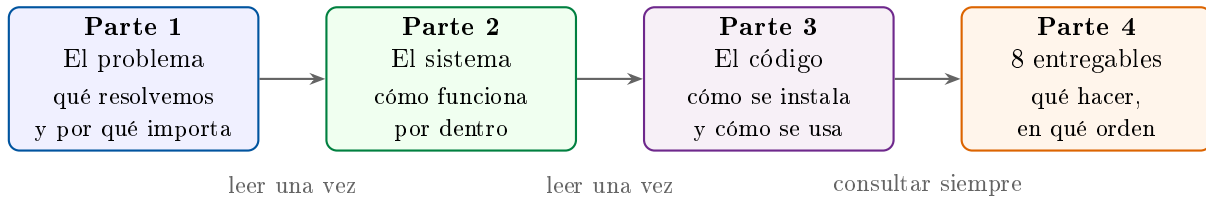
Cómo usar esta guía	4
Estructura del documento	4
Reglas de trabajo	4
Qué se entrega en cada sesión	4
Mapa de los ocho entregables	5
Protocolo de errores	6
Por qué hay que desconfiar de los números que salen bien	6
Qué hacer si detectas un problema	7
Las tres partes deliberadamente incompletas	7
Parte 1: Qué problema resolvemos	9
La situación de partida	9
Cómo se leen las intenciones del cerebro	9
Dónde está el problema	9
El momento crítico: cuando el usuario cambia de mirada	10
Qué hace la literatura y qué proponemos nosotros	10
La idea en una frase	10
Cómo se distinguen IC y TR	11
Qué aporta este trabajo	11
Parte 2: Cómo funciona el sistema	12
Las dos etapas de una tarea	12
Los cuatro comandos	12
El enclavamiento: la clave para entender todo lo demás	12
Las dos ventanas de análisis	13
Por qué la racha protege al sistema	14
La máquina de estados	14
Cómo se mueve el robot: los tres escenarios	14
La Etapa 2: elegir un objeto dividiendo la pantalla	15
Cuándo pasa de la Etapa 1 a la Etapa 2	16
Cómo se distinguen objetos y obstáculos	17
Las dos máquinas	17
Parte 3: El código	19
Qué recibes	19
Instalación	19
La regla más importante: config.py	20
La configuración se valida sola	20
Cómo se usa cada paquete	21
Trabajar sin hardware: el generador de señal simulada	22
El contrato: la pieza que lo mantiene todo unido	22
Las tres partes que tienes que completar	23
Sobre el amplificador g.tec en particular	23
El bloque reservado para la Fase 2	23
Cómo se convierte una orden en movimiento	24
Por qué hace falta un controlador y no basta con ordenar	24
El antiwindup: un detalle pequeño con consecuencias grandes	25
Dónde está en el código	26
Un segundo controlador, más simple, para el final	26

Cómo están organizadas las pruebas	26
Estructura de carpetas recomendada	26
Parte 4: Los ocho entregables	28
Entregable 1: Entorno de trabajo y núcleo del sistema	29
Objetivo	29
Por qué importa	29
Tareas	29
Checklist	30
Qué entregar	30
Preguntas para pensar	30
Entregable 2: Estímulo visual y verificación temporal	31
Objetivo	31
Por qué importa	31
Tareas	31
Checklist	32
Qué entregar	32
Preguntas para pensar	32
Entregable 3: Adquisición de EEG y conexión del amplificador	33
Objetivo	33
Por qué importa	33
Tareas	33
Checklist	34
Qué entregar	35
Preguntas para pensar	35
Entregable 4: Capa de control y despliegue en el robot	36
Objetivo	36
Por qué importa	36
Tareas	36
Checklist	38
Qué entregar	38
Preguntas para pensar	38
Entregable 5: Visión, marcadores y selección de objetos	39
Objetivo	39
Por qué importa	39
Tareas	39
Checklist	41
Qué entregar	41
Preguntas para pensar	41
Entregable 6: Conexión del módulo de detección de estados	42
Objetivo	42
Por qué importa	42
Tareas	42
Checklist	43
Qué entregar	43
Preguntas para pensar	44

Entregable 7: Ejecución de experimentos y análisis	45
Objetivo	45
Por qué importa	45
Los dos experimentos	45
Tareas	45
Checklist	47
Qué entregar	47
Preguntas para pensar	47
Entregable 8: Informe integrado	48
Objetivo	48
Por qué importa	48
Contenido del informe	48
Sobre la sección 3	48
Checklist	49
Qué entregar	49
Resumen: las diez cosas que no hay que olvidar	50

Cómo usar esta guía

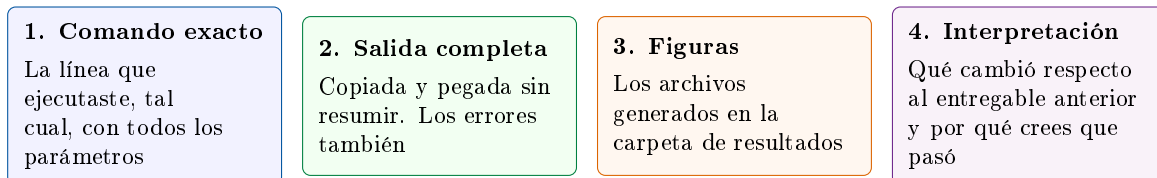
Estructura del documento



Reglas de trabajo

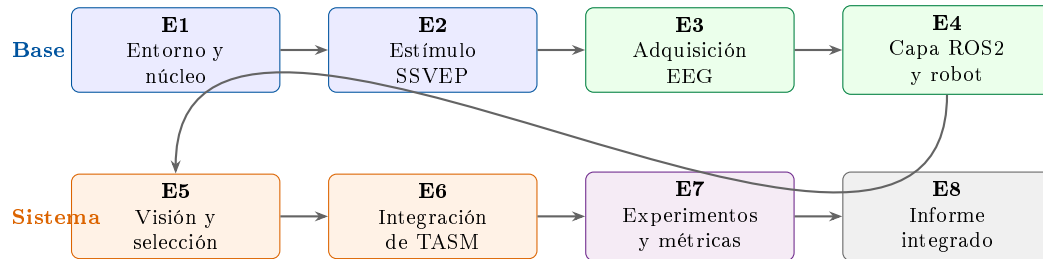
1. **Los entregables se numeran de forma única.** Cuando escribas al profesor, di siempre el número: “Entregable 3”, no “la parte del EEG”.
2. **Cada entregable produce evidencia documentada.** Salida de consola completa, figuras generadas, y **una interpretación escrita por ti** de lo que significan los números. Sin interpretación, el entregable no está completo.
3. **No hay fechas.** Se avanza por entregables. En cada sesión se revisa lo completado desde la anterior.
4. **Si encuentras un error, intenta arreglarlo primero.** Documenta el cambio y entrega el código modificado. El protocolo está detallado en la sección siguiente.
5. **Un cambio a la vez.** Si modificas dos cosas y el resultado cambia, no sabrás cuál fue.
6. **Guarda todo.** Cada ejecución que produzca números debe quedar guardada. Repetir un experimento porque no se guardó el anterior es la forma más común de perder tiempo.

Qué se entrega en cada sesión



El punto 4 es el que importa. Los otros tres los produce el ordenador. La interpretación la produces tú, y es lo que se discute en la sesión.

Mapa de los ocho entregables



El orden importa. Los entregables 1 y 2 son obligatorios antes que nada: si el núcleo no está validado y el estímulo no es correcto, ningún resultado posterior es interpretable.

A partir del 3 hay algo de flexibilidad, pero la secuencia está pensada para que cada entregable use lo aprendido en el anterior.

Protocolo de errores

EL CÓDIGO QUE RECIBES ES UN BORRADOR.

No contiene errores puestos a propósito: no es un ejercicio de buscar el fallo escondido. Se ha probado, y las 279 pruebas automáticas pasan.

Pero **no se ha ejecutado nunca en el hardware del laboratorio**, y partes de él están deliberadamente incompletas porque dependen de librerías que no estaban disponibles.

Tu revisión es real, no un juego.

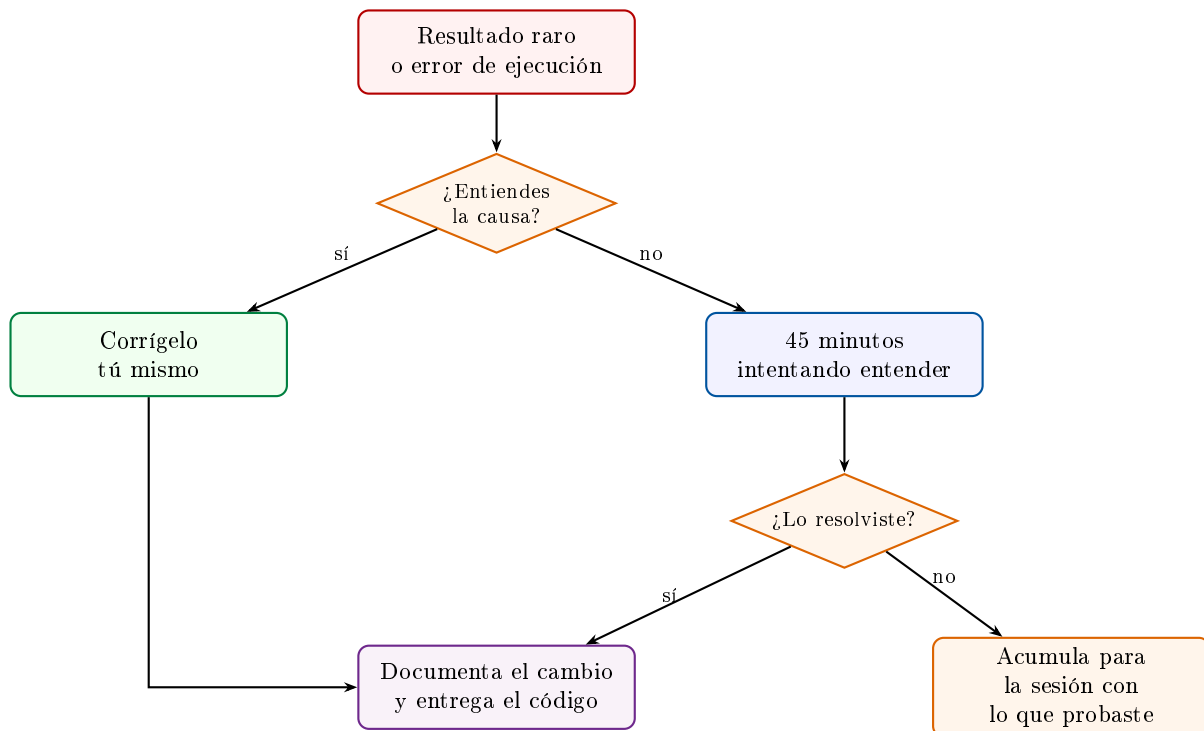
Por qué hay que desconfiar de los números que salen bien

Durante el desarrollo aparecieron ocho errores. **Cuatro de ellos no produjeron ningún mensaje de error.** El programa corría, generaba figuras y devolvía números perfectamente creíbles.

Error	Síntoma visible	Cómo se detectó
El filtro notch eliminaba un armónico útil de la señal	Ninguno	Una comprobación automática al cargar la configuración
La métrica principal contaba las protecciones del sistema como si fueran errores del usuario	La diferencia entre las dos condiciones del experimento salía nula	Inspeccionar una por una las transiciones contadas
El generador de señal simulada usaba un modelo físicamente incorrecto	Idem: diferencia nula	Calcular a mano la probabilidad de que ocurriera lo que se quería medir
El umbral de rechazo de artefactos era demasiado estricto	Se descartaban ventanas de señal perfectamente válidas	Una ventana “limpia” de la demostración salía marcada como inválida
El mismo archivo se importaba dos veces por rutas distintas	Un mensaje absurdo: “el tipo es X, debería ser X”	Leer el mensaje con atención en vez de asumir que era un fallo tonto

Lección práctica: cuando un resultado sale sospechosamente bien o sospechosamente mal, esa es la señal para revisar el código, no para celebrar ni para rendirse.

Qué hacer si detectas un problema



Formato del reporte de cambio. En un archivo de texto junto al código:

1. Archivo y función modificados
2. Qué hacía antes y qué hace ahora
3. Por qué crees que estaba mal
4. Qué resultado cambia con la corrección, con números antes y después

Entrega el paquete completo, no solo el archivo cambiado.

Por qué se hace así. Si cada duda pequeña sube al profesor, el proyecto se atasca. Se espera que intentes resolverlo.

Si no puedes en 45 minutos, para y acumula: llegar a la sesión con “probé A, B y C, y esto fue lo que pasó” vale mucho más que llegar con “no funciona”.

Las tres partes deliberadamente incompletas

Parte	Por qué falta	Qué tienes que hacer
Amplificador g.tec	Su librería solo existe para Windows y necesita licencia	Completar las llamadas marcadas con >>> COMPLETAR usando la librería que tengáis en el laboratorio (Entregable 3)
Detector YOLO	La librería no estaba instalada	Instalar ultralytics y completar dos funciones. El sistema funciona sin él (Entregable 5)
Módulo TASM	Lo desarrolla otra persona del equipo	Conectar el adaptador cuando esté disponible (Entregable 6)

En los tres casos está definido el contrato: qué debe devolver cada función y en qué unidades. Rellenar las llamadas dentro de ese contrato es trabajo acotado, y el resto del sistema funciona sin cambios.

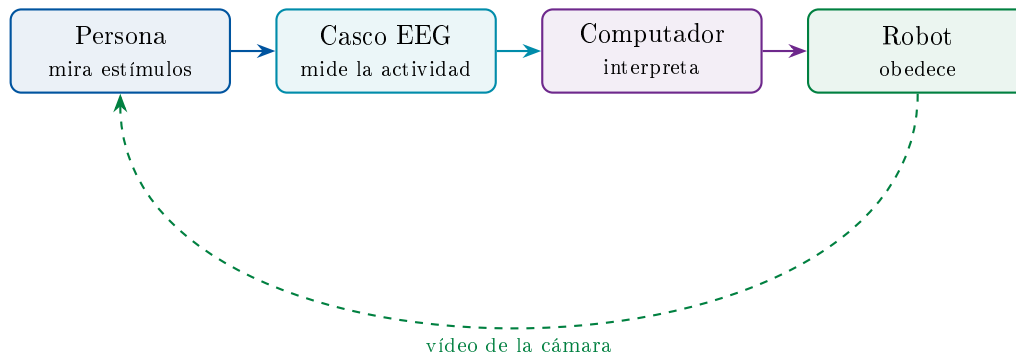
Lo que **no** debes hacer es cambiar el contrato para que encaje con lo que devuelve la librería. La conversión va dentro del adaptador.

Parte 1: Qué problema resolvemos

La situación de partida

Imagina una persona con una discapacidad motora severa que no puede mover los brazos ni las piernas, pero conserva el control de la mirada. Queremos que pueda manejar un robot móvil a distancia: llevarlo por una habitación, esquivar obstáculos, y elegir un objeto entre varios.

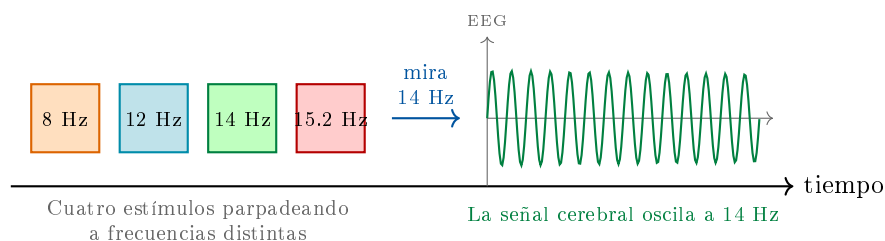
Todo eso usando **únicamente señales de su cerebro**.



Cómo se leen las intenciones del cerebro

Usamos un fenómeno llamado **SSVEP**, siglas en inglés de *potencial evocado visual de estado estable*.

La idea es sorprendentemente simple. Si pones delante de alguien una luz que parpadea a 14 veces por segundo y esa persona la mira fijamente, su corteza visual empieza a oscilar **a esa misma frecuencia**. Esa oscilación se puede medir con electrodos en la nuca.



Entonces, si ponemos cuatro estímulos parpadeando a frecuencias distintas y cada uno significa un comando (girar a la izquierda, girar a la derecha, avanzar, parar), basta con detectar *qué frecuencia* aparece en el EEG para saber qué quiere el usuario.

Dónde está el problema

Todo lo anterior funciona bien cuando el sistema le dice al usuario *cuándo* mirar. Eso se llama sistema **síncrono**, y es como funcionan los teclados cerebrales: el sistema marca los turnos.

Pero conducir un robot no funciona así. El usuario decide cuándo quiere dar una orden. Eso es un sistema **asíncrono**, y añade una pregunta que antes no existía.

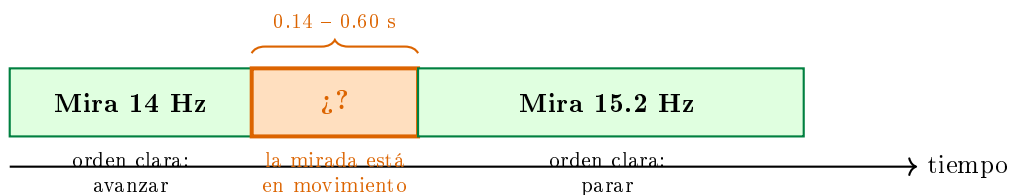
	Síncrono	Asíncrono
¿Cuándo mira el usuario?	El sistema se lo indica	Cuando él quiere
Instante de inicio	Conocido	Desconocido
Pregunta a resolver	¿Cuál frecuencia?	¿Hay orden? y ¿cuál?
Aplicación típica	Deletreo	Robot, silla de ruedas

La pregunta “¿hay orden?” parece fácil pero no lo es, y ahí está el problema que este trabajo ataca.

El momento crítico: cuando el usuario cambia de mirada

Aquí viene el punto central de todo el proyecto. Léelo con calma.

Cuando el usuario deja de mirar un estímulo y pasa a mirar otro, hay un intervalo de medio segundo aproximadamente en que **la señal cerebral es una mezcla**: todavía queda respuesta del estímulo que abandona, y ya empieza a aparecer la del nuevo.



El problema: durante ese intervalo, un detector que solo sepa distinguir “hay orden” de “no hay orden” ve una señal parcial y frecuentemente la interpreta como **una orden válida**.

El robot recibe entonces un comando que el usuario nunca quiso dar. Y como el comando se queda enganchado (ver más adelante), el robot lo mantiene hasta que el usuario consiga corregirlo, lo cual requiere *otra* mirada, y por tanto otra oportunidad de error.

Qué hace la literatura y qué proponemos nosotros

Enfoque	Cómo funciona	Por qué no nos sirve
Periodo muerto programado	Tras cada orden, el sistema apaga los estímulos un segundo y tira esos datos	Presupone que el sistema <i>sabe</i> cuándo ocurre la transición, porque él la programa. En asíncrono no lo sabe
Interruptor con otra señal	Se añaden electrodos que detectan un parpadeo o una contracción muscular	Requiere instrumentación extra y una acción motora voluntaria, difícil para las personas a las que va dirigido
Nuestra propuesta	Detectar la transición desde la propia señal SSVEP , por cómo cambia en el tiempo	—

La idea en una frase

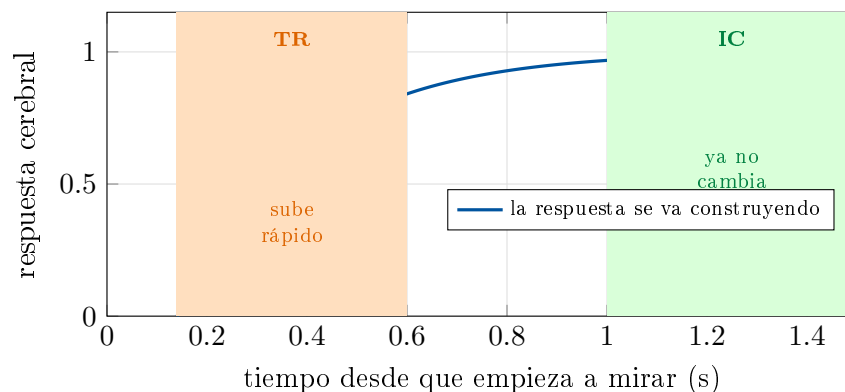
En lugar de dos estados (“hay orden” / “no hay orden”), usamos **tres**:

IC	<i>Intentional Control</i>	El usuario mira un estímulo y quiere dar esa orden
TR	<i>Transición</i>	Está moviendo la mirada. No hay orden que respetar
Idle	<i>Reposo</i>	No mira ningún estímulo. Está observando la escena

El módulo que hace esa distinción se llama **TASM** (*Transition-Aware State Monitor*) y lo desarrolla otra persona del equipo. Tu trabajo es el sistema robótico que lo usa.

Cómo se distinguen IC y TR

No por la *amplitud* de la señal, que en ambos casos es intermedia, sino por su **derivada**: en TR la respuesta está *subiendo*; en IC ya se estabilizó.



Idea central en una frase: IC y TR se distinguen mal por *cuánta* respuesta hay, pero se distinguen bien por *si está creciendo*.

Qué aporta este trabajo

Ningún sistema publicado detecta la transición de mirada a partir de la propia señal SSVEP, sin electrodos adicionales y sin periodos muertos programados.

Y ninguno lo integra como una **compuerta** sobre la lógica de comandos de un robot móvil, que es lo que impide que una orden accidental quede enganchada.

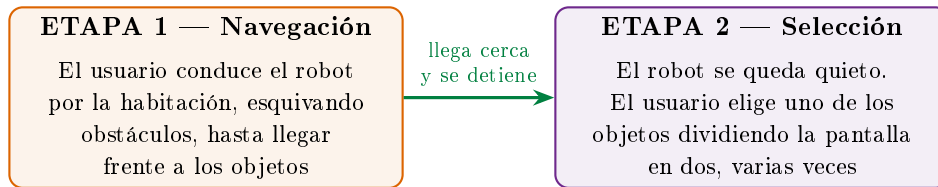
Tres cosas que NO son novedad, y conviene que lo tengas claro para no atribuirles importancia que no tienen:

- Dividir la pantalla para seleccionar entre muchos objetos con pocas frecuencias. Existe desde 2010.
- Asignar la frecuencia según la posición en pantalla en lugar de según el objeto. Existe desde 2021.
- Combinar BCI con seguimiento visual para acercarse a un objeto. Existe desde 2017.

Parte 2: Cómo funciona el sistema

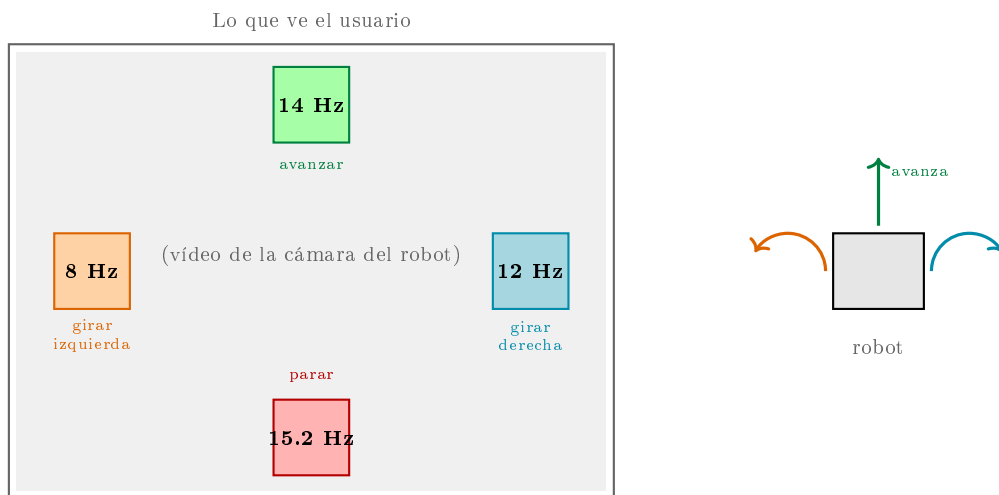
Las dos etapas de una tarea

Una tarea completa tiene dos fases bien separadas.



Los cuatro comandos

En la Etapa 1 el usuario dispone de cuatro órdenes, cada una asociada a una frecuencia y a una posición en la pantalla.



La posición de cada estímulo coincide con su significado: el de girar a la izquierda está a la izquierda, el de avanzar arriba. Eso hace que el usuario no tenga que memorizar nada.

¿Por qué no hay comando de retroceder? Porque el TurtleBot3 puede girar sobre sí mismo. Su diagonal mide 0.415 m y el paso más estrecho de los escenarios es de 0.70 m, así que siempre tiene sitio para darse la vuelta.

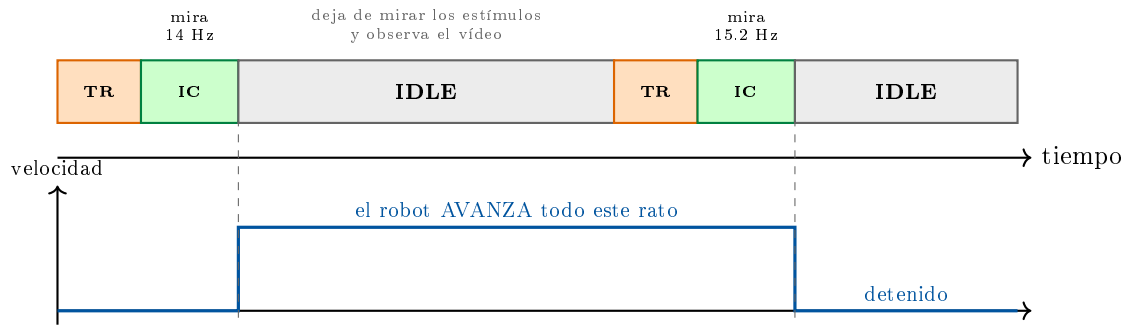
Además, si detecta un obstáculo muy cerca, retrocede automáticamente sin que el usuario tenga que pedirlo.

El enclavamiento: la clave para entender todo lo demás

Esta es la idea que más cuesta al principio, y de ella dependen muchas decisiones del sistema. Léela dos veces si hace falta.

El usuario NO mantiene la mirada fija mientras el robot se mueve.

La secuencia real es esta:



El usuario mira un estímulo unos instantes, el comando **se engancha** (se “enclava”), y entonces **deja de mirar los estímulos** para observar el vídeo de la cámara y ver por dónde va.

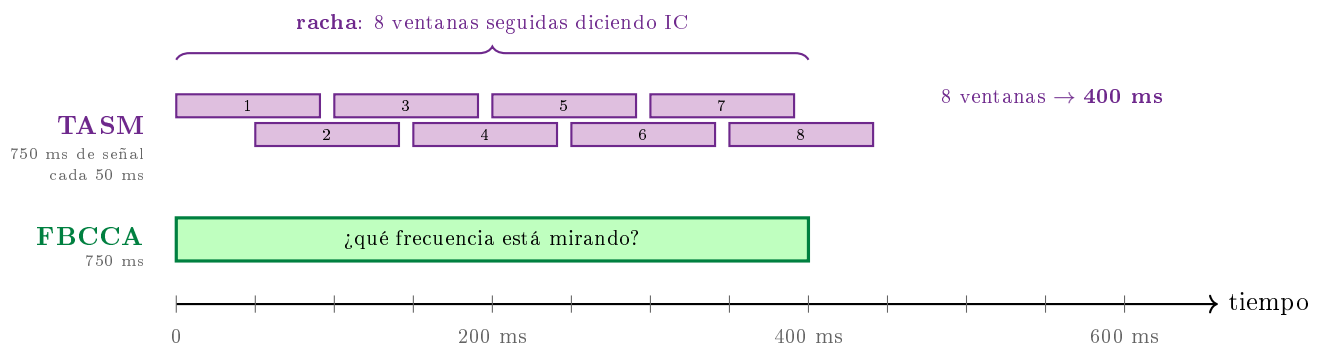
El robot mantiene la orden hasta recibir otra.

Consecuencia importante: durante la mayor parte del tiempo el usuario está en estado Idle, y eso es **lo normal**, no un fallo. El robot debe seguir obedeciendo la última orden.

Si el robot se detuviera cada vez que el usuario deja de mirar los estímulos, el sistema sería inútil: habría que mirar fijamente durante todo el trayecto.

Las dos ventanas de análisis

Aquí hay una confusión frecuente que conviene aclarar de entrada. El sistema responde **dos preguntas distintas, en paralelo**, sobre la misma señal.

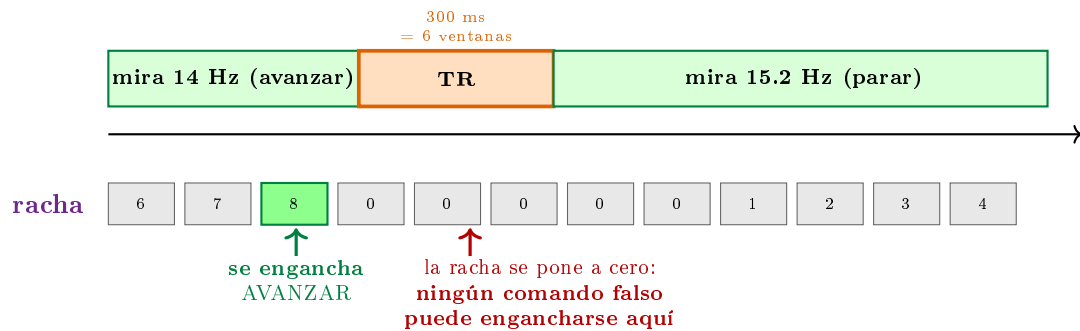


	TASM	FBCCA
Pregunta	¿El usuario está mirando <i>algún</i> estímulo?	¿ <i>Cuál</i> de los cuatro está mirando?
Respuesta	IC, TR o Idle	Una de las cuatro frecuencias
Ventana	750 ms, avanza cada 50 ms	750 ms

Lo importante: las dos duran lo mismo y terminan a la vez. La racha de 8 ventanas de TASM ocupa $8 \times 50 = 400$ ms.

Así que la racha **no añade ninguna espera**: al cabo de ese tiempo el sistema tiene las dos respuestas simultáneamente, y con ambas engancha el comando.

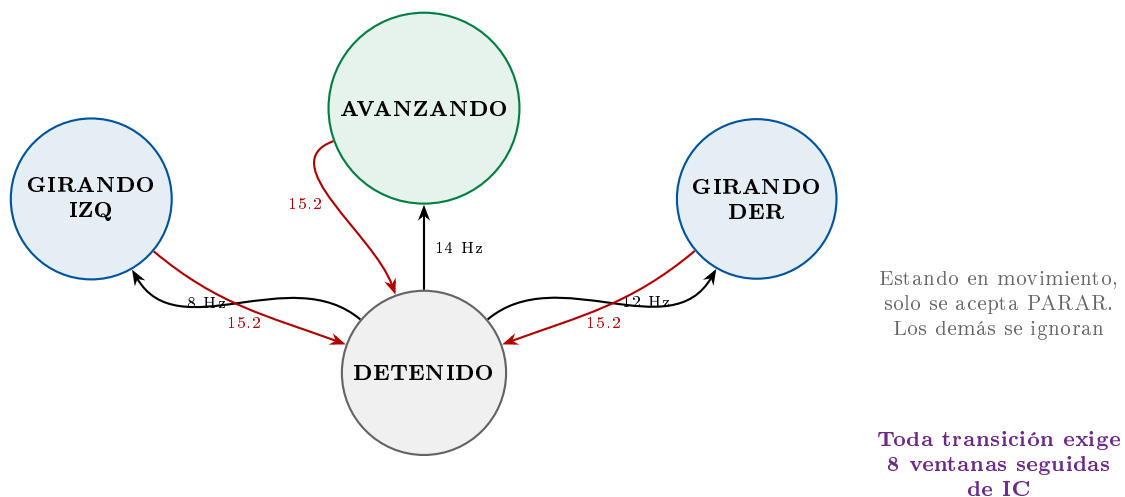
Por qué la racha protege al sistema



Una transición de mirada dura entre 3 y 12 ventanas (140 a 600 ms). La racha exige 8. Por aritmética simple, una transición típica **no alcanza** a completarla.

Esa es la protección estructural del sistema: aunque el clasificador se equivoque varias veces durante la transición, el comando falso no llega a engancharse.

La máquina de estados



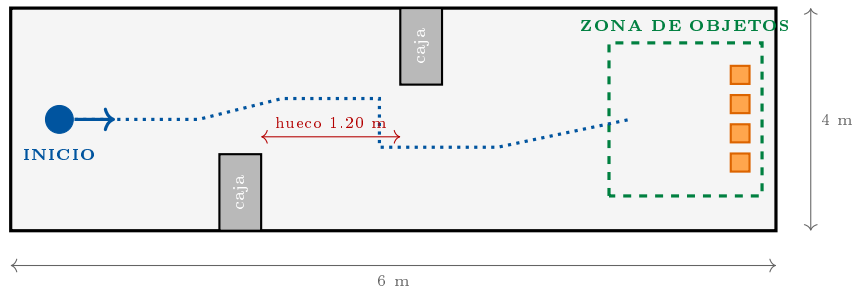
Para cambiar de acción hay que pasar por DETENIDO. Si el robot está avanzando y quieres girar, primero paras y luego giras.

Eso evita cambios bruscos, y tiene un efecto secundario útil para el experimento: cada cambio de acción genera **dos** transiciones de mirada en lugar de una, lo que da más eventos que medir.

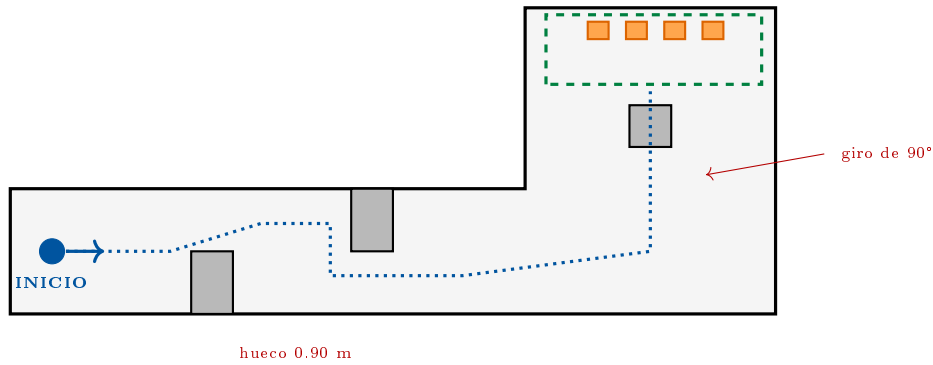
Cómo se mueve el robot: los tres escenarios

Los experimentos usan tres recorridos de dificultad creciente. La dificultad se controla con un solo número: el **hueco libre** entre obstáculos.

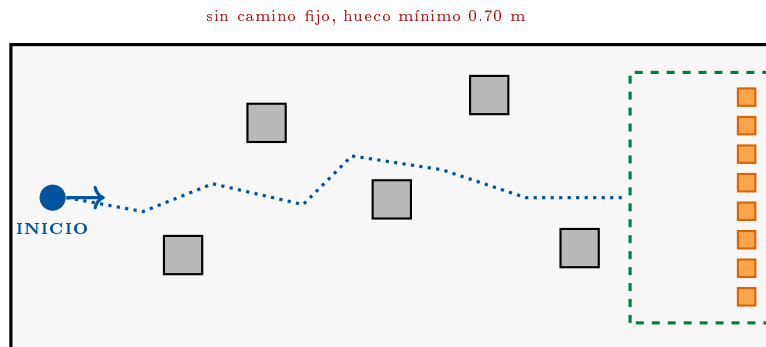
Escenario 1 — Corredor recto (fácil)



Escenario 2 — Corredor en L (medio)



Escenario 3 — Sala abierta (difícil)

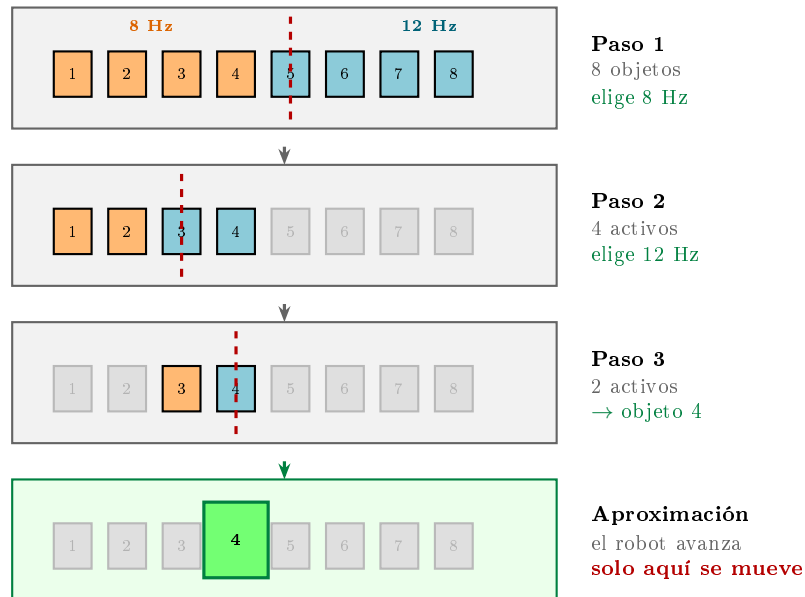


Escenario	Obstáculos	Hueco	Holgura real	Dificultad
1 — Corredor recto	2	1.20 m	0.92 m	baja
2 — Corredor en L	3	0.90 m	0.62 m	media
3 — Sala abierta	5	0.70 m	0.42 m	alta

Table 1: La holgura real es el hueco menos el ancho del robot (0.281 m). Es lo que mide de verdad la dificultad.

La Etapa 2: elegir un objeto dividiendo la pantalla

Cuando el robot llega frente a los objetos, se detiene y cambia la interfaz. Ahora solo hay dos frecuencias, y los objetos se dividen en dos grupos.



La cámara no se mueve durante los pasos 1 a 3. La imagen es exactamente la misma; lo único que cambia es *cuáles* objetos parpadean.

Por eso los objetos no se hacen más pequeños al ir dividiendo, y por eso el número de decisiones es exactamente $\lceil \log_2 N \rceil$: 3 decisiones para 8 objetos, 2 para 4, 1 para 2.

N objetos	Decisiones	Ancho de la fila	Distancia del robot	Tamaño en píxeles
2	1	0.45 m	0.37 m	573 px
4	2	0.95 m	0.79 m	268 px
8	3	1.95 m	1.62 m	131 px

Table 2: Objetos de 20 cm separados 25 cm. La separación es siempre la misma; lo que cambia es a qué distancia se detiene el robot.

¿Por qué 2, 4 y 8 y no 3, 5 o 6? Porque con potencias de dos todos los objetos requieren exactamente el mismo número de decisiones.

Con 6 objetos, algunos se alcanzan en 2 pasos y otros en 3. Eso mezclaría dos situaciones distintas bajo la misma etiqueta al comparar tiempos.

Cuándo pasa de la Etapa 1 a la Etapa 2

Hacen falta **tres condiciones a la vez**:

1. Distancia

Al menos 2 objetos
a menos de 1.8 m

2. Camino libre

Nada entre el robot
y los objetos

3. Permanencia

El robot lleva
2 segundos parado

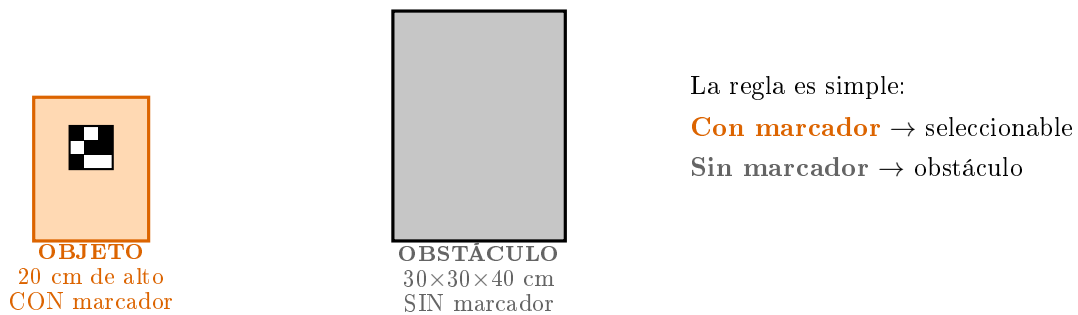
La tercera es la que más importa. Sin ella, el sistema interpretaría como “he llegado” cualquier parada que el usuario haga a mitad de camino para mirar alrededor.

Además, el contador se pone a cero si el robot se mueve. Pararse un segundo, avanzar un poco

y volver a parar no acumula.

El cambio es **automático** (no hay que confirmarlo: el usuario ya mostró su intención al detenerse frente a los objetos) y **reversible** (mirando el estímulo de parar se vuelve a la Etapa 1).

Cómo se distinguen objetos y obstáculos

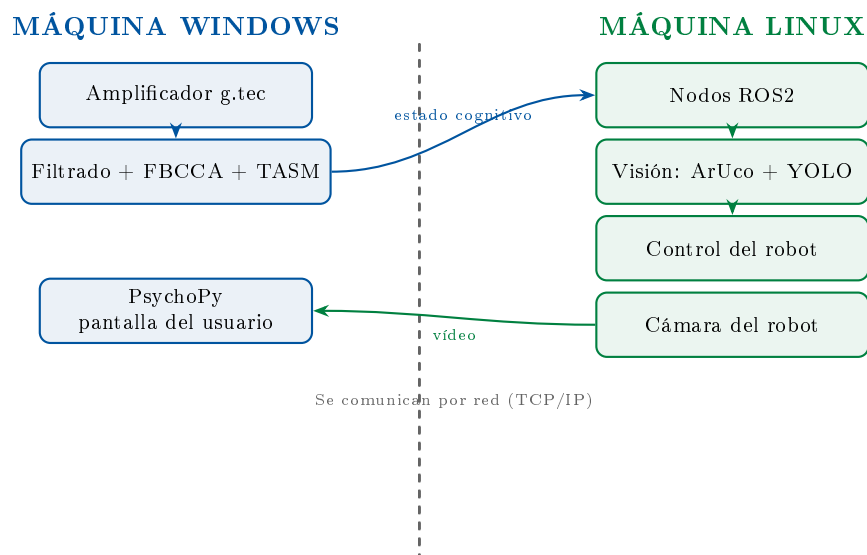


Los obstáculos deben medir al menos 25 cm de alto. No es un capricho: el sensor láser del robot escanea en un plano a unos 18 cm del suelo. Un obstáculo más bajo sería **invisible** para el sistema de seguridad.

Por eso se usan cajas de 40 cm, que dan margen de sobra.

Las dos máquinas

El sistema corre en dos ordenadores porque ninguno puede hacerlo todo.



	Windows	Linux
Por qué	La librería del amplificador g.tec solo existe para Windows	ROS2 necesita Linux
Qué corre	EEG, filtrado, clasificación, pantalla del usuario	Robot, visión, control

Un detalle de diseño que conviene entender: el vídeo de la cámara y los estímulos parpadeantes son **independientes**. Los estímulos se generan con el contador de fotogramas de la pantalla, no con la llegada del vídeo.

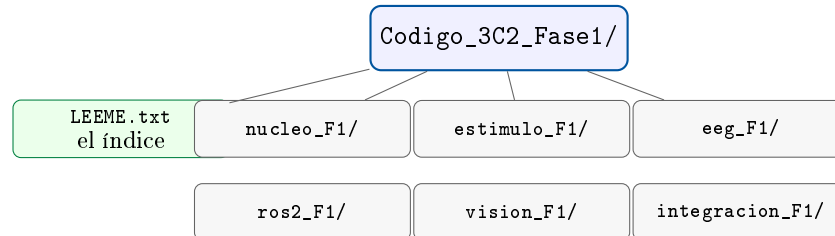
Consecuencia: si la red se atasca, la imagen se congela pero la señal SSVEP sigue siendo correcta. Si estuvieran acoplados, cada paquete perdido corrompería el estímulo.

Parte 3: El código

Qué recibes

Un único archivo comprimido: `Codigo_3C2_Fase1_v1.0.zip`

Al descomprimirlo encontrarás **seis paquetes de Python** y un archivo `LEEME.txt` que sirve de índice. Cada paquete se puede usar por separado y tiene su propio `README.txt`.



Empieza por LEEME.txt. Es corto y te dice qué paquete usa cada entregable.

Paquete	Pruebas	Qué hace	Necesita
nucleo_F1	50	Máquina de estados, clasificador, simulador	Solo Python y NumPy
estimulo_F1	44	Genera los estímulos parpadeantes y la pantalla	PsychoPy (solo para ver la pantalla)
eeg_F1	44	Lee el EEG, lo filtra, lo trocea en ventanas	La librería del amplificador
ros2_F1	48	Los nodos que controlan el robot	ROS2 (solo para el robot real)
vision_F1	52	Detecta objetos y decide cuándo cambiar de etapa	OpenCV
integracion_F1	41	Conecta con TASM y ejecuta los experimentos	Solo Python

Buena noticia: los paquetes 1, 4 y 6 funcionan en cualquier ordenador sin instalar nada especial. Puedes avanzar mucho antes de tocar hardware.

Notarás que algunos archivos están repetidos en varios paquetes: `config.py` aparece en los seis, y hay módulos duplicados en las carpetas `logica/`.

Es deliberado. Así cada paquete corre por sí solo, sin depender de que los demás estén instalados: puedes trabajar en el Entregable 3 sin haber tocado todavía los de visión o ROS2.

`config.py` es el mismo archivo y va creciendo: el de `integracion_F1` es el más completo porque incluye las secciones de los seis bloques. Si lo modificas, hazlo en el paquete donde estés trabajando y anota el cambio.

Instalación

```
# Lo basico, para todos los paquetes
pip install numpy scipy pytest
```

```
# Para vision (OJO: contrib, no opencv-python a secas)
pip install opencv-contrib-python

# Solo en la maquina del laboratorio
pip install psychopy
pip install ultralytics
```

Atención con OpenCV. Tiene que ser `opencv-contrib-python`, no `opencv-python`. El módulo de marcadores ArUco está solo en la versión contrib.

Si te equivocas, el error será `module 'cv2' has no attribute 'aruco'`.

La regla más importante: config.py

Todos los números del sistema viven en un solo archivo: config.py.

Ningún otro módulo define constantes propias. Todos reciben la configuración como parámetro.

Si cambias un valor ahí, el cambio se propaga a todo el sistema. No hay manera de que un módulo lo sobrescriba en silencio.

```
from config import CONFIG

# LEER: siempre asi
fs = CONFIG.bci.fs # frecuencia de muestreo
frecuencias = CONFIG.bci.frecuencias # las cuatro frecuencias

# ESCRIBIR: imposible, lanza una excepcion
CONFIG.bci.fs = 512.0 # FrozenInstanceError

# PROBAR UNA VARIANTE sin tocar el archivo
cfg = CONFIG.copia_con(bci=dict(n_conf=4))
```

Por qué está protegido contra escritura. Si un módulo pudiera cambiar un parámetro global en caliente, el sistema dejaría de ser reproducible: dos ejecuciones idénticas darían resultados distintos y sería imposible saber por qué.

La configuración se valida sola

Al importar `config.py` se ejecuta una comprobación automática. Si algo es incoherente, avisa.

```
python config.py
```

Eso vuelca todos los parámetros y ejecuta la validación. Ya ha detectado errores reales durante el desarrollo:

Qué detectó	Por qué importaba
Que con 16 objetos el robot tendría que estar a 3.27 m	La sala solo tiene 4 m de profundidad: nunca se habría podido hacer
Que el filtro de red eliminaba un armónico útil	Se perdía información de la señal sin que nada fallara
Que el bloque de lectura era mayor que el paso de ventana	Se habrían perdido actualizaciones del estado

Si al importar aparece un aviso, léelo. No es ruido: alguien lo puso ahí porque detectaba un problema real.

Cómo se usa cada paquete

Todos siguen el mismo patrón: un archivo `run_*.py` que es el punto de entrada, con varios subcomandos.

Paquete 1: `nucleo_F1`

```
python run.py config      # vuelca y valida la configuracion
python run.py smoke      # prueba rapida de todos los modulos
python run.py test       # las 50 pruebas automaticas
python run.py fsm        # la maquina de estados suprime durante TR
python run.py fbcca      # el clasificador recupera la frecuencia
python run.py seleccion  # la seleccion requiere log2(N) decisiones
python run.py trial      # un recorrido completo simulado
python run.py viabilidad # compara las dos condiciones del experimento
python run.py barrido    # prueba varios valores de la racha
```

Paquete 2: `estimulo_F1`

```
python run_estimulo.py config      # parametros del estimulo
python run_estimulo.py realizable  # por que sinusoidal y no cuadrada
python run_estimulo.py espectro    # la senal generada es correcta
python run_estimulo.py deriva      # el problema del reloj, con numeros
python run_estimulo.py timing      # verificacion principal
python run_estimulo.py demo        # ver la pantalla (necesita PsychoPy)
```

Paquete 3: `eeg_F1`

```
python run_eeg.py config      # parametros de adquisicion
python run_eeg.py filtros     # la cadena deja pasar lo que debe
python run_eeg.py causal      # por que el filtrado es como es
python run_eeg.py pipeline    # verificacion principal
python run_eeg.py latencia    # el computo cabe en el tiempo disponible
python run_eeg.py impedancias  # comprobacion previa a cada sesion
```

Paquete 4: `ros2_F1`

```
python run_ros2.py grafo      # como se conectan los nodos
python run_ros2.py seguridad  # el sistema de proteccion
python run_ros2.py navegacion # la asistencia de evasion
python run_ros2.py mision     # verificacion principal
python run_ros2.py integracion # simula todo el sistema sin ROS2
python run_ros2.py despliegue # instrucciones para el robot
```

Paquete 5: `vision_F1`

```
python run_vision.py marcadores # genera los marcadores para imprimir
python run_vision.py deteccion  # detecta sobre imagenes sinteticas
python run_vision.py alcance    # precision segun la distancia
python run_vision.py transicion # verificacion principal
python run_vision.py aproximacion # el robot se acerca al objeto
python run_vision.py calibracion # como calibrar la camara
```

Paquete 6: integracion_F1

```
python run_integracion.py contrato      # verificacion principal
python run_integracion.py regresion     # compara dos fuentes de senal
python run_integracion.py protocolo     # el protocolo de sesion completo
python run_integracion.py expA          # experimento A simulado
python run_integracion.py expB          # experimento B simulado
python run_integracion.py migracion     # como conectar el modulo real
```

Trabajar sin hardware: el generador de señal simulada

Puedes probar **todo el sistema** sin casco EEG, sin sujeto y sin robot. El código incluye un generador que produce estados cognitivos sintéticos con errores realistas.

```
config.tasm.fuente = "mock"           # simulado, para desarrollar
config.tasm.fuente = "dataset"        # senal real grabada
config.tasm.fuente = "real"           # senal real en vivo
```

El generador simulado es una herramienta de desarrollo. Sirve para construir y probar el sistema, no para producir resultados.

Cuando llegue el momento de los experimentos de verdad, la señal tiene que ser EEG real.

El contrato: la pieza que lo mantiene todo unido

Todos los componentes que producen o consumen estado cognitivo hablan el mismo idioma:

```
estado      # "IC", "TR" o "Idle"
freq_idx    # cual de las 4 frecuencias (0 a 3), o -1 si no aplica
p_max       # confianza sobre CUAL frecuencia, entre 0 y 1
lambda_bci  # confianza sobre si HAY orden, entre 0 y 1
valido      # false si la ventana tenia artefactos
```

Ese contrato se definió **antes** de que existiera el módulo TASM, y por eso cambiar de señal simulada a señal real es cambiar un parámetro.

Si cada parte hubiera inventado su propio formato, conectarlas ahora sería un trabajo de semanas.

Cuando conectes el módulo real, no cambies el contrato. Si ese módulo devuelve los datos en otro formato, la conversión va *dentro* del adaptador, no fuera.

Hay un verificador que comprueba esto automáticamente y detecta cinco errores típicos que **no dan ningún mensaje de error**:

- Los índices empiezan en 1 en vez de en 0
- Un valor viene en porcentaje (0–100) en vez de entre 0 y 1
- El orden de las frecuencias no coincide
- El estado viene como número en vez de como texto
- El módulo nunca reporta el estado TR

Las tres partes que tienes que completar

Parte	Dónde	Qué hacer	Entregable
Amplificador	<code>eeg_source.py</code> , clase <code>FuenteGUSBamp</code>	Abrir el dispositivo, configurarlo, leer bloques	E3
Detector YOLO	<code>vision_fusion.py</code> , clase <code>DetectorYOLO</code>	Cargar el modelo y hacer la inferencia	E5
Módulo TASM	<code>tasm_interface.py</code> , clase <code>FuenteTASMReal</code>	Cargar el modelo y convertir su salida al contrato	E6

Las tres están marcadas en el código con el mismo formato:

```
def iniciar(self):
    # >>> COMPLETAR: abrir el dispositivo
    #
    #     import pygds
    #     self._dev = pygds.GDS()
    #     self._dev.SamplingRate = self.cfg.bci.fs
    #     ...
    #
    raise NotImplementedError(...)
```

El comentario incluye un esbozo de lo que hay que escribir. **No es código que funcione**: es una guía de qué llamadas hacer y en qué orden. La librería concreta que tengáis puede tener nombres distintos.

Sobre el amplificador g.tec en particular

La librería de g.tec es propietaria, solo existe para Windows, y su interfaz cambia entre versiones. Por eso el código no la incluye.

Lo que sí está definido es el contrato: el método `leer()` debe devolver un array de forma `(n_muestras, n_canales)` en **microvoltios**, con los canales en el orden exacto de `config.bci.canales`.

Dos cosas que hay que verificar y casi nadie verifica:

1. Las unidades. Si la librería devuelve voltios en lugar de microvoltios, todos los umbrales de artefactos estarían mal por un factor de un millón. Y no daría ningún error.

2. El orden de los canales. El sistema espera exactamente:

Pz, P05, P03, P0z, P04, P06, Oz, O1, O2

Si los electrodos están conectados en otro orden, nada falla de forma visible: el sistema sigue corriendo y dando resultados, pero peores.

El bloque reservado para la Fase 2

En `ctrl_node.py` hay un fragmento delimitado así:

```
===== INICIO BLOQUE FASE 2 -- NO ACTIVAR EN FASE 1 =====
...
===== FIN BLOQUE FASE 2 =====
```

Está desactivado por un parámetro en `params.yaml`:

```
usar_lambda_bci: false
```

Qué hace ese bloque. En la Fase 1 el controlador es sencillo: la velocidad de referencia entra tal cual. La Fase 2 propone ponderarla por `lambda_bci`, la confianza continua sobre si el usuario está dando una orden. Cuando esa confianza baja, el control prioriza una trayectoria suave sobre seguir agresivamente un comando incierto.

Por qué está ahí ya, si no se usa.

El valor `lambda_bci` **ya viaja** en los mensajes y **ya se guarda** en cada recorrido, aunque no intervenga en el control.

Esa decisión no es accidental: si no se registrara ahora, los datos que recojas en la Fase 1 **no servirían** para desarrollar la Fase 2 más adelante, y habría que repetir todas las sesiones con sujetos.

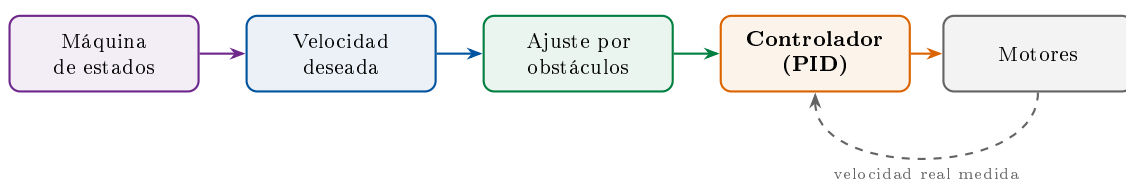
Qué tienes que hacer tú con ese bloque: nada.

Déjalo desactivado. No lo borres ni lo actives. Si lo activaras, los resultados de tus experimentos no serían comparables con lo previsto para esta fase.

Solo tienes que saber que existe, y no sorprenderte al verlo en el código.

Cómo se convierte una orden en movimiento

Cuando la máquina de estados decide “avanzar”, eso todavía no mueve nada. Hay que traducirlo a velocidades concretas y conseguir que el robot las alcance de verdad.



Por qué hace falta un controlador y no basta con ordenar

Si le pides al robot “ve a 0.15 m/s”, no llega instantáneamente a esa velocidad. Tiene inercia, las ruedas patinan un poco, el suelo tiene irregularidades, y la batería se va descargando.

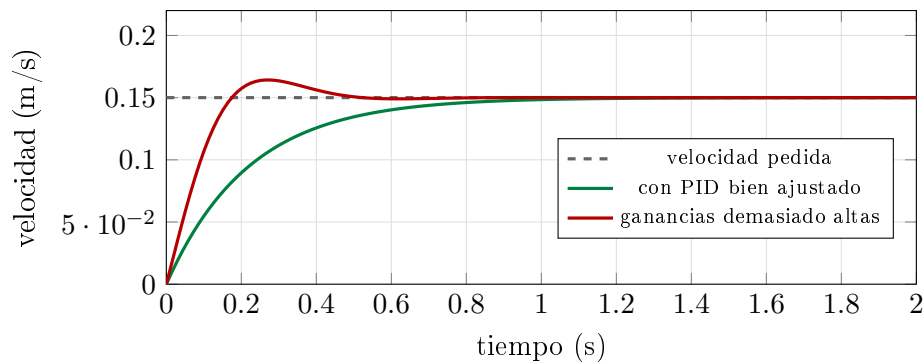
El controlador compara **lo que pediste** con **lo que el robot está haciendo realmente** (medido por la odometría) y corrige la diferencia.

Esa diferencia se llama **error**:

$$e(t) = v_{\text{deseada}} - v_{\text{real}}$$

El PID calcula la corrección combinando tres términos, uno por cada letra del nombre:

Término	Ganancia	Qué mira	Qué aporta
Proporcional	$k_p = 1.20$	El error <i>ahora</i>	Cuanto más lejos estés de la velocidad pedida, más fuerte corrige
Integral	$k_i = 0.10$	El error <i>acumulado</i>	Corrige desviaciones pequeñas pero persistentes, por ejemplo si el robot se queda siempre un poco corto
Derivativo	$k_d = 0.05$	Cómo <i>cambia</i> el error	Amortigua. Frena la corrección cuando ve que ya se está acercando, para no pasarse



Si las ganancias son demasiado altas, el robot **se pasa** de la velocidad pedida y oscila alrededor de ella. Si son demasiado bajas, tarda mucho en alcanzarla.

Los valores del código son un punto de partida razonable, pero **pueden necesitar ajuste** en el robot real: dependen del peso, del suelo y del estado de la batería.

El antiwindup: un detalle pequeño con consecuencias grandes

El término integral acumula el error. Eso es útil, pero tiene un problema cuando el robot *no puede* alcanzar la velocidad pedida.

Imagina que el usuario ordena avanzar, pero el sistema de evasión está frenando el robot porque hay un obstáculo cerca. El error se mantiene alto durante segundos, y el término integral **crece sin parar**.

Cuando el obstáculo desaparece, ese valor acumulado se descarga de golpe y el robot **sale disparado** muy por encima de la velocidad pedida.

La solución es poner un tope al valor que puede acumular el integrador. Se llama *antiwindup* y en el código es el parámetro `antiwindup = 1.0`.

Es una línea de código que se olvida con muchísima facilidad, y su efecto **solo aparece en situaciones concretas**: el robot funciona perfectamente durante las pruebas normales y da un acelerón inexplicable justo después de esquivar algo.

Hay una prueba automática que lo verifica: satura el controlador durante 500 ciclos y comprueba que el integrador no supera el tope.

Dónde está en el código

Qué	Dónde
La lógica del PID	<code>ros2_F1/logica/nodos_logica.py</code> , clase <code>LogicaControl</code>
Las ganancias	<code>config.py</code> , sección <code>ControlConfig</code>
El nodo que lo ejecuta	<code>ctrl_node.py</code> , a 20 Hz
Las pruebas	<code>tests/test_ros2.py</code> , clase <code>TestControl</code>

Un detalle sobre el ritmo. El controlador corre a 20 veces por segundo, el mismo ritmo al que llegan las decisiones del sistema BCI. No es casualidad: si el control fuera más lento, perdería órdenes; si fuera más rápido, calcularía correcciones sobre información repetida.

Un segundo controlador, más simple, para el final

Cuando el usuario ya ha elegido un objeto y el robot se acerca a él, se usa un controlador **distinto y más sencillo**: solo proporcional, sin los términos integral ni derivativo.

Por qué: la maniobra es corta y termina con el robot parado. Un término integral acumularía error durante la aproximación y produciría un sobreimpulso justo al final, que es el peor momento posible cuando te estás acercando a un objeto.

Ese controlador hace dos cosas en orden: primero alinea el robot con el objeto, y solo cuando está bien orientado avanza. Avanzar mientras se está muy desalineado alejaría al robot de la trayectoria útil.

Cómo están organizadas las pruebas

Cada paquete tiene una carpeta `tests/`. Se ejecutan así:

```
python -m pytest tests/ -v
```

Cuando una prueba falla, no la silencies ni cambies el criterio para que pase. Cada prueba comprueba una propiedad que el sistema debe cumplir.

Si falla, o bien el código tiene un error, o bien la configuración es incoherente. Ambas cosas hay que arreglarlas, no ocultarlas.

Al reportar un fallo, indica **qué prueba** falla y pega la salida completa de `pytest`. Eso convierte una conversación difusa en un dato.

Estructura de carpetas recomendada

```
~/proyecto/
nucleo_F1/
estimulo_F1/
eeg_F1/
ros2_F1/
vision_F1/
integracion_F1/
resultados/          <- todo lo que generes va aqui
```

```
E1/  
E2/  
...  
reportes/          <- tus interpretaciones escritas
```

Guarda todo. Cada ejecución que produzca números debe quedar registrada con la fecha y el comando exacto que la generó.

Repetir un experimento porque no se guardó el anterior es la forma más común de perder tiempo en un proyecto de investigación.

Parte 4: Los ocho entregables

Cada entregable tiene la misma estructura:

- **Objetivo** — qué se busca conseguir
- **Por qué importa** — para qué sirve en el conjunto
- **Tareas** — qué hacer, paso a paso
- **Checklist** — casillas que marcar antes de darlo por hecho
- **Qué entregar** — los archivos y el escrito
- **Preguntas para pensar** — para la sesión de revisión

ENTREGABLE 1

Entorno de trabajo y núcleo del sistema

Objetivo

Tener el entorno funcionando, entender la máquina de estados con enclavamiento, y comprobar que la propiedad central del sistema se cumple: que una transición de mirada no engancha comandos.

Por qué importa

Todo lo demás depende de esto. Si la máquina de estados no hace lo que debe, ningún resultado posterior significa nada.

Además, este entregable es donde aprendes a leer el código: el resto de paquetes siguen la misma estructura.

Tareas

1.1 Instalar y verificar

```
pip install numpy scipy pytest
cd nucleo_F1
python run.py config      # vuelca la configuracion
python run.py smoke      # prueba rapida
python run.py test       # las 50 pruebas
```

Al ejecutar `config` aparecerá un aviso sobre la racha de confirmación. **No es un error**: es un aviso de diseño que la configuración emite a propósito. Léelo y trata de entender qué dice.

1.2 Leer `config.py` entero

Es largo pero está comentado. No hace falta que entiendas cada fórmula, pero sí que sepas responder:

- ¿Cuáles son las cuatro frecuencias y qué comando es cada una?
- ¿Cuántos canales de EEG y en qué orden?
- ¿Cuánto mide la ventana de análisis y cada cuánto avanza?
- ¿Cuántas ventanas seguidas hacen falta para enganchar un comando?

1.3 Comprobar la propiedad central

```
python run.py fsm
```

Esto verifica tres cosas. Léelas en la salida y entiende cada una:

1. Con IC sostenido, el comando se engancha tras 8 ventanas
2. Con transición de mirada, aunque el clasificador se equivoque, no se engancha nada
3. Estando en movimiento, solo se acepta la orden de parar

1.4 Comprobar la selección de objetos

```
python run.py seleccion
```

Fíjate en la columna “uniforme”. Con 2, 4 y 8 objetos todos los objetivos requieren el mismo número de decisiones. Eso es lo que hace que la afirmación de complejidad logarítmica sea verificable.

1.5 Correr el simulador completo

```
python run.py trial
```

Compara las dos condiciones. Anota los números.

1.6 El barrido de la racha

```
python run.py barrido
```

Este comando prueba varios valores de la racha de confirmación. **Es el más importante del entregable** y sobre él va tu interpretación escrita.

Verás un compromiso: con una racha muy corta entran comandos falsos; con una muy larga ninguna transición alcanza a completarla y el problema desaparece (pero también la posibilidad de medir la solución).

Checklist

Comprobación
☐ Las 50 pruebas pasan
☐ <code>run.py smoke</code> dice TODO CORRECTO
☐ Sé decir de memoria las cuatro frecuencias y su significado
☐ Entiendo por qué la racha son 8 ventanas y no otra cifra
☐ <code>run.py fsm</code> supera las tres verificaciones
☐ <code>run.py seleccion</code> da “uniforme = SI” para 2, 4 y 8
☐ He corrido el barrido y guardado la tabla
☐ Puedo explicar el compromiso de la racha con mis palabras

Qué entregar

1. Salida completa de los seis comandos
2. La tabla del barrido
3. **Media página** explicando qué significa el compromiso de la racha: qué pasa si es muy corta, qué pasa si es muy larga, y por qué el valor actual está donde está

Preguntas para pensar

1. Si el usuario deja de mirar los estímulos durante 10 segundos, ¿qué hace el robot? ¿Por qué está diseñado así?
2. ¿Por qué hay que pasar por DETENIDO para cambiar de acción?
3. En el barrido, ¿por qué crees que con racha muy larga la diferencia entre las dos condiciones desaparece?

ENTREGABLE 2

Estímulo visual y verificación temporal

Objetivo

Entender por qué el estímulo se genera como se genera, y verificar que las cuatro frecuencias salen exactamente donde deben.

Por qué importa

Si el estímulo no es correcto, **nada de lo que venga después vale**. Da igual lo bueno que sea el clasificador: si la pantalla no parpadea a la frecuencia que crees, estarás midiendo otra cosa.

Y el error más común en esta parte **no da ningún síntoma**: el programa corre, la pantalla parpadea, y la frecuencia va derivando poco a poco.

Tareas

2.1 Entender por qué no se puede usar parpadeo simple

```
cd estimulo_F1
python run_estimulo.py realizable
```

La pantalla refresca 60 veces por segundo. Si encendemos y apagamos el estímulo, solo podemos generar frecuencias que sean $60/n$ con n par.

Ninguna de las cuatro frecuencias del sistema cumple eso. Por eso se modula el **brillo** de forma continua en lugar de encender y apagar.

2.2 Verificar el espectro

```
python run_estimulo.py espectro
```

Comprueba que el error es 0.000 Hz en las cuatro frecuencias.

2.3 El error del reloj

```
python run_estimulo.py deriva
```

Este comando es el más instructivo del entregable.

Simula lo que pasa si se calcula el estímulo usando el tiempo transcurrido en lugar del contador de fotogramas. Con un error de solo 0.1 milisegundos por fotograma, la señal se desfasa 265 grados en medio minuto.

En un experimento de cinco minutos, la señal estaría completamente descorrelacionada de la referencia. Y no daría ningún error.

2.4 Buscar el patrón correcto en el código

Abre `stimulus.py` y localiza dónde se calcula el brillo. Comprueba que usa el número de fotograma, no el reloj:

```
# CORRECTO: usa el contador de fotogramas
angulo = 2*pi*f*(frame/refresh) + fase

# MAL: usa el reloj del sistema
angulo = 2*pi*f*time.time()
```

2.5 La verificación principal

```
python run_estimulo.py timing
python run_estimulo.py test
```

2.6 Ver la pantalla (si tienes PsychoPy)

```
python run_estimulo.py demo
```

Comprueba que el monitor va realmente a 60 Hz. El programa lo mide al arrancar y avisa si no coincide.

Si el monitor fuera de 144 Hz y el código asume 60, todas las frecuencias estarían mal por un factor de 2.4 sin que nada fallara.

Checklist

Comprobación
☐ Las 44 pruebas pasan
☐ Sé explicar por qué no se usa parpadeo simple
☐ Las cuatro frecuencias caen en su bin con error 0.000 Hz
☐ He visto la tabla de deriva y entiendo el problema
☐ He localizado en el código dónde se calcula el brillo
☐ <code>timing</code> da COMPUERTA SUPERADA
☐ Si tengo PsychoPy: he visto la pantalla y el monitor va a 60 Hz

Qué entregar

1. Salida de los cinco comandos
2. La tabla de deriva
3. **Media página** explicando por qué se usa el contador de fotogramas y qué pasaría si no

Preguntas para pensar

1. ¿Por qué la ventana de análisis es de 5 segundos y no de 4?
2. El estímulo tiene el 60% de los píxeles encendidos, no el 100%. ¿Qué se gana con eso? ¿Qué ventaja añadida tiene en este sistema concreto?
3. Si el monitor fuera de 75 Hz, ¿qué habría que cambiar?

ENTREGABLE 3

Adquisición de EEG y conexión del amplificador

Objetivo

Entender la cadena de procesamiento de la señal, y **conectar el amplificador real**.

Por qué importa

Aquí hay dos errores clásicos que casi todo el mundo comete la primera vez, y que producen resultados que parecen buenos en análisis pero no se reproducen en tiempo real.

Este entregable existe sobre todo para que los entiendas.

Tareas

3.1 Los dos errores clásicos

```
cd eeg_F1
python run_eeg.py causal
```

Error 1: usar un filtro que mira al futuro.

La función `filtfilt` es la que casi todo el mundo usa para filtrar EEG porque no distorsiona la fase. Lo consigue filtrando dos veces: hacia adelante y hacia atrás.

Y ahí está el problema: para calcular la salida en el instante t , el pase hacia atrás necesita muestras *posteriores* a t . En análisis offline eso es trivial porque toda la señal ya está grabada. En tiempo real esas muestras no existen todavía.

El comando lo demuestra: filtra un escalón y mide cuánto responde *antes* de que ocurra.

Error 2: no conservar la memoria del filtro.

Un filtro tiene memoria: su salida depende de lo anterior. Si se procesa cada bloque de datos por separado sin conservar esa memoria, cada bloque arranca desde cero y produce un transitorio artificial.

A 21 bloques por segundo, eso serían 21 transitorios por segundo contaminando la señal.

El comando mide ambas variantes: con memoria el error es exactamente cero; sin ella, alcanza el 95% del rango de la señal.

3.2 El filtro de red

```
python run_eeg.py filtros
```

Fíjate en el punto crítico: el cuarto armónico de 15.2 Hz cae en 60.8 Hz, a menos de un hercio de la red eléctrica.

Por eso el filtro de red tiene que ser especialmente estrecho. Con el valor habitual se comería

ese armónico, y perderíamos información útil sin que nada fallara.

3.3 La verificación principal

```
python run_eeg.py pipeline
python run_eeg.py latencia
python run_eeg.py test
```

3.4 Conectar el amplificador

Esta es la parte principal del entregable.

Abre `eeg_source.py` y localiza la clase `FuenteGUSBamp`. Hay cuatro puntos marcados >>> COMPLETAR.

Lo que hay que hacer:

1. **Importar la librería y abrir el dispositivo.** La forma habitual con este amplificador es un módulo de Python que se distribuye con el equipo. Pregunta en el laboratorio cuál es y qué versión tenéis.
2. **Configurarlo:** frecuencia de muestreo, canales activos, y **desactivar los filtros del propio aparato**. El filtrado se hace en software para tener control exacto de la fase.
3. **Implementar la lectura.** Debe devolver un bloque de datos sin bloquear el programa.
4. **Implementar la medida de impedancias**, si la librería lo permite.

Dos verificaciones que no se pueden saltar:

Las unidades. Comprueba si la librería devuelve microvoltios o voltios. Si son voltios, hay que multiplicar por un millón. Si te equivocas, todos los umbrales de rechazo de artefactos estarán mal por ese factor, y no dará ningún error.

El orden de los canales. Tiene que ser exactamente: Pz, P05, P03, P0z, P04, P06, 0z, 01, 02. Si los electrodos están conectados en otro orden, el sistema seguirá funcionando pero peor, sin avisar.

3.5 Verificar con el amplificador conectado

```
python run_eeg.py impedancias      # todas por debajo de 5 kOhm
python run_eeg.py pipeline         # con fuente = "gusbamp"
```

Checklist

Comprobación
☐ Las 44 pruebas pasan
☐ Sé explicar por qué <code>filtfilt</code> no sirve en tiempo real
☐ Sé explicar por qué el filtro necesita memoria entre bloques
☐ Entiendo por qué el filtro de red es más estrecho de lo habitual
☐ He completado las cuatro llamadas del amplificador
☐ He verificado las unidades (microvoltios)
☐ He verificado el orden de los canales
☐ Las impedancias bajan de 5 kOhm
☐ El pipeline corre con señal real

Qué entregar

1. El archivo `eeg_source.py` con el amplificador conectado
2. Salida de los comandos, con señal real si es posible
3. Reporte de impedancias
4. **Una página** explicando: cómo verificaste las unidades, cómo verificaste el orden de canales, y qué problemas encontraste al conectar el aparato

Preguntas para pensar

1. El sistema distingue dos tipos de retraso: el tiempo de cálculo y el retraso total. ¿Por qué solo uno tiene que caber en el periodo de ventana?
2. El umbral para rechazar artefactos no está fijado a mano sino calculado. ¿De dónde sale ese cálculo y por qué es mejor que un número fijo?
3. ¿Qué pasaría si conectaras los electrodos en orden alfabético en lugar del orden especificado?

ENTREGABLE 4

Capa de control y despliegue en el robot

Objetivo

Entender cómo se decide qué velocidad enviar al robot, y desplegar el sistema en el TurtleBot3.

Por qué importa

Aquí vive el orden de prioridades del sistema, que es lo que garantiza que la seguridad nunca quede supeditada a la interpretación de la señal cerebral.

Tareas

4.1 Entender la estructura

```
cd ros2_F1
python run_ros2.py grafo
```

Fíjate en que hay **dos** componentes que pueden mandar velocidad al robot: el de control normal y el de seguridad.

Es redundancia a propósito. Si el primero se colgara o tuviera un error, el robot quedaría sin protección. El de seguridad es deliberadamente simple para que haya poco que pueda fallar.

4.2 El sistema de seguridad

```
python run_ros2.py seguridad
```

Fíjate en la **histéresis**: se entra en emergencia a 15 cm pero no se sale hasta superar los 20 cm durante cinco ciclos seguidos.

Sin eso, un obstáculo justo en el umbral haría oscilar el sistema varias veces por segundo, produciendo un movimiento a tirones.

Un detalle importante: el componente de seguridad **no sabe nada** del estado cognitivo del usuario, y no debe saberlo.

Un obstáculo a 15 cm es igual de peligroso tanto si el usuario está dando una orden como si no. Además, acoplarlos introduciría un modo de fallo nuevo: un error del detector podría desactivar la protección.

4.3 La asistencia de evasión

```
python run_ros2.py navegacion
```

El sistema no navega solo: el usuario decide adónde va. Lo que hace el campo potencial es corregir la trayectoria para que no roce.

Verificación importante: el signo. Si al probarlo en el robot real ves que se *pega* a las paredes en lugar de evitarlas, el signo está invertido. Es un error muy fácil de cometer al derivar el gradiente.

4.4 El controlador

Abre `logica/nodos_logica.py` y localiza la clase `LogicaControl`. Comprueba que reconoces los tres términos del PID y la línea del antiwindup.

```
python -m pytest tests/test_ros2.py::TestControl -v
```

Fíjate en la prueba `test_antiwindup_acota_el_integrador`: satura el controlador durante 500 ciclos y verifica que el integrador no crece sin límite.

Prueba a comentar la línea del antiwindup y vuelve a ejecutarla. Verás fallar la prueba, y el valor que alcanza el integrador. Luego descoméntala.

4.5 La verificación principal

```
python run_ros2.py mision
python run_ros2.py integracion
python run_ros2.py test
```

4.6 Desplegar en el robot

```
python run_ros2.py despliegue # lee las instrucciones primero
```

Luego, en la máquina Linux:

```
cd ~/proyecto/ros2_ws
colcon build --packages-select turtlebot3_bci_3c2
source install/setup.bash

# Verificar que los mensajes se generaron
ros2 interface show turtlebot3_bci_3c2/msg/TASMState

# Probar sin robot
ros2 launch turtlebot3_bci_3c2 test_sin_robot.launch.py

# En otra terminal
ros2 topic hz /tasm/state # deberia dar ~20 Hz
ros2 topic echo /bci/command
```

4.7 Con el robot

```
ros2 launch turtlebot3_bci_3c2 bci_navigation.launch.py \
  windows_ip:=<la IP de la maquina Windows>
```

El nodo de seguridad arranca **primero y sin retraso**. Si el robot ya está encendido y hay un obstáculo cerca, queremos protección desde el primer instante.

Checklist

Comprobación
☐ Las 48 pruebas pasan
☐ Sé explicar el orden de prioridades y por qué es ese
☐ Entiendo qué es la histéresis y para qué sirve
☐ Entiendo por qué el nodo de seguridad ignora el estado cognitivo
☐ Sé qué hace cada uno de los tres términos del PID
☐ Entiendo qué es el antiwindup y qué pasa sin él
☐ He probado a desactivar el antiwindup y he visto fallar la prueba
☐ <code>mission</code> da COMPUERTA SUPERADA
☐ El paquete compila con <code>colcon build</code>
☐ <code>ros2 interface show</code> muestra los mensajes
☐ Los topics publican a la frecuencia esperada
☐ Verifiqué el signo del campo potencial en el robot real

Qué entregar

1. Salida de los comandos de verificación
2. Salida de `ros2 topic hz` para los topics principales
3. Un vídeo corto del robot moviéndose (aunque sea con señal simulada)
4. **Una página:** qué problemas hubo al desplegar, cómo verificaste el signo del campo potencial, y qué observaste del comportamiento real frente al simulado

Preguntas para pensar

1. Si se pierde la comunicación con la máquina Windows, ¿qué hace el robot? ¿Por qué no mantiene la última orden?
2. ¿Por qué los mensajes van con transporte “best-effort” en lugar de garantizado? ¿Cuál es la excepción y por qué?
3. ¿Qué pasaría si el nodo de seguridad corriera más lento que el de control?
4. El controlador de navegación es un PID completo, pero el de aproximación final solo usa el término proporcional. ¿Por qué esa diferencia?
5. Si al probar en el robot real vieras que oscila alrededor de la velocidad pedida, ¿qué ganancia ajustarías y en qué dirección?

ENTREGABLE 5

Visión, marcadores y selección de objetos

Objetivo

Preparar el montaje físico (marcadores, objetos, obstáculos), calibrar la cámara, y verificar que el cambio de etapa funciona.

Por qué importa

De la distancia medida por la cámara depende cuándo el sistema decide que ha llegado. Si esa medida está mal, el cambio de etapa ocurrirá donde no debe.

Y hay dos cosas que casi nadie mide bien y que invalidan todas las distancias.

Tareas

5.1 Generar e imprimir los marcadores

```
cd vision_F1
python run_vision.py marcadores
```

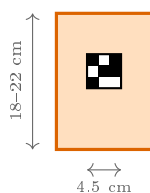
Esto genera un archivo de imagen con los marcadores.

AL IMPRIMIR: elige “tamaño real” o “100%”, **nunca** “ajustar a página”. Las impresoras escalan por defecto.

DESPUÉS DE IMPRIMIR: mide con una regla el lado del marcador, sin contar el borde blanco. Si no coincide con el valor de la configuración, corrige la configuración con el valor real.

Un error del 5% aquí produce un error del 5% en **todas** las distancias que el sistema calcule.

5.2 Preparar los objetos



Requisitos de los objetos:

- Altura entre 18 y 22 cm
- Base estable (que no se vuelquen)
- Visualmente distintos entre sí
- Superficie plana para el marcador (si es curvo, pega antes un cartón)

Pueden ser objetos variados: muñecos, figuras, botellas. No hace falta que sean uniformes en forma, solo en altura.

De hecho es preferible: da naturalidad a la escena.

5.3 Preparar los obstáculos

Propiedad	Valor	Por qué
Dimensiones	30 × 30 × 40 cm	—
Altura mínima	25 cm	El láser escanea a 18 cm del suelo. Más bajo sería invisible
Superficie	Opaca, mate	El vidrio y el plástico brillante dan lecturas erráticas
Color	Gris o cartón	Que el usuario los distinga de los objetos
Marcador	Ninguno	Esa ausencia es lo que los identifica como obstáculos

5.4 Calibrar la cámara

```
python run_vision.py calibracion
```

Los valores actuales de la cámara son **aproximados**, deducidos del campo visual declarado. La distorsión de las lentes pequeñas es apreciable y produce error sistemático.

El comando da el procedimiento completo y el código de referencia. Hay que hacerlo con un patrón de ajedrez y unas 20 fotos.

Cómo saber si quedó bien: coloca un marcador a exactamente 1.00 m medido con cinta métrica, y comprueba que el sistema reporta esa distancia con menos de 2 cm de error.

Repítelo a 0.5 y 1.5 m. Si el error crece de forma sistemática con la distancia, el problema está en el tamaño del marcador, no en la calibración.

5.5 Verificar la detección

```
python run_vision.py deteccion
python run_vision.py alcance
```

Fíjate en cómo crece el error con la distancia. Es esperado: a más distancia, el marcador ocupa menos píxeles, y un error de un píxel en las esquinas se amplifica en la distancia calculada.

5.6 La verificación principal

```
python run_vision.py transicion
python run_vision.py test
```

Comprueba que las tres condiciones se evalúan bien y que ninguna se puede saltar.

5.7 Completar el detector YOLO (opcional pero recomendado)

```
pip install ultralytics
```

Abre `vision_fusion.py` y completa la clase `DetectorYOLO`. Hay dos puntos marcados.

El sistema funciona sin YOLO. La identidad y la distancia las da el marcador. Lo que se pierde es que el recuadro parpadeante encuadre el objeto completo en lugar de una estimación.

5.8 Montar la escena y medir

Verificación que solo se puede hacer con el robot: la cámara del TurtleBot3 va montada baja, a unos 15–20 cm del suelo.

Hay que medir a qué altura queda el centro de la imagen a 1.8 m de distancia, y colocar los marcadores ahí. Si los objetos apoyados en el suelo quedan fuera del encuadre, la solución es **elevarlos sobre una plataforma**, no mover la cámara.

Checklist

Comprobación
☐ Las 52 pruebas pasan
☐ Marcadores impresos a tamaño real
☐ He medido el marcador impreso con una regla
☐ Objetos preparados: 18–22 cm, base estable, distinguibles
☐ Obstáculos de al menos 25 cm, sin marcador
☐ Cámara calibrada y verificada con cinta métrica
☐ transicion da COMPUERTA SUPERADA
☐ He verificado la altura del marcador en el robot real

Qué entregar

1. Fotos del montaje: objetos, obstáculos, marcadores pegados
2. Los parámetros de calibración obtenidos
3. **Tabla de verificación:** distancia real medida con cinta contra distancia reportada por el sistema, a 0.5, 1.0 y 1.5 m
4. Salida de los comandos
5. **Media página:** qué error de distancia obtuviste y si es aceptable

Preguntas para pensar

1. ¿Por qué se usan dos detectores (marcadores y YOLO) en lugar de uno?
2. De las tres condiciones para cambiar de etapa, ¿cuál evita el error más probable y por qué?
3. ¿Por qué el número de objetos es 2, 4 y 8 y no 3, 5 o 6?

ENTREGABLE 6

Conexión del módulo de detección de estados

Objetivo

Conectar el módulo TASM real (cuando esté disponible) y verificar que la integración no ha roto nada.

Por qué importa

Cuando se cambie la señal simulada por la real, los resultados van a cambiar. La pregunta es **por qué** cambiaron.

Hay dos causas posibles y hay que poder distinguirlas:

1. El módulo real se comporta distinto del simulado. Es lo esperado.
2. La integración rompió algo.

Sin una verificación sistemática, un problema del segundo tipo aparecería como “el módulo funciona peor de lo esperado”, y se buscaría el fallo en el sitio equivocado durante semanas.

Tareas

6.1 Entender el contrato

```
cd integracion_F1
python run_integracion.py config
python run_integracion.py contrato
```

El comando **contrato** simula cinco adaptadores mal escritos y comprueba que el verificador los detecta. **Ninguno de esos cinco errores daría un mensaje de error** en ejecución normal. Léelos con atención: son los errores que tú podrías cometer al conectar el módulo real.

6.2 Completar el adaptador

Abre `tasm_interface.py` y localiza `FuenteTASMReal`. Hay dos puntos marcados.

La conversión es el punto crítico. Verifica campo por campo:

- Si el módulo devuelve el estado como número (0, 1, 2), hay que traducirlo
- Si los índices de frecuencia empiezan en 1, hay que restar 1
- Si algún valor viene en porcentaje, hay que dividir por 100
- El orden de las frecuencias tiene que coincidir con la configuración

No cambies el contrato para que encaje. La conversión va dentro del adaptador.

6.3 Verificar antes de nada

```
config.tasm.fuente = "dataset"
```

```
python run_integracion.py contrato
```

No sigas hasta que esto diga CONTRATO CORRECTO. Si el contrato está mal, todo lo demás dará números que no significan lo que crees.

6.4 Verificación de que nada se rompió

```
python run_integracion.py regresion
```

Este comando **no compara valores** (que cambian legítimamente al pasar de simulado a real), sino **propiedades que deben cumplirse siempre**:

1. La máquina de estados no engancha comandos durante una transición
2. Sí engancha tras las 8 ventanas de mirada sostenida
3. La selección sigue requiriendo el número exacto de decisiones
4. El módulo produce las tres clases

Si alguna se rompe, el problema es de integración, no del módulo.

6.5 Qué esperar que cambie

Métrica		Qué esperar	Por qué
Errores de transición	durante	Probablemente <i>suba</i>	El simulado usa valores de laboratorio; en vivo será peor
Comandos enganchados	falsos	Depende	Si los errores vienen en racha, subirá más de lo esperado
Decisiones de selección		No debe cambiar	Es determinista

El último punto es el más útil para diagnosticar: **si el número de decisiones deja de ser el esperado, hay un problema de integración**, no del módulo.

6.6 Migración completa

```
python run_integracion.py migracion # los cinco pasos, en orden
```

Checklist

Comprobación
☐ Las 41 pruebas pasan
☐ Entiendo los cinco errores que detecta el verificador
☐ He completado el adaptador
☐ contrato da CONTRATO CORRECTO con la fuente real
☐ regresion no reporta propiedades violadas
☐ El número de decisiones de selección no cambió
☐ He documentado qué conversiones hicieron falta

Qué entregar

1. El archivo `tasm_interface.py` con el adaptador completo
2. Salida de `contrato` y `regresion`

3. **Una página:** qué conversiones hicieron falta, qué cambió respecto a la señal simulada, y si las propiedades estructurales se mantuvieron

Preguntas para pensar

1. ¿Por qué hay un paso intermedio con datos grabados en lugar de pasar directamente de simulado a sujeto real?
2. De los cinco errores que detecta el verificador, ¿cuál te parece más peligroso y por qué?
3. ¿Por qué el número de decisiones de selección sirve como diagnóstico?

ENTREGABLE 7

Ejecución de experimentos y análisis

Objetivo

Ejecutar los dos experimentos con sujetos y analizar los resultados.

Por qué importa

Es donde se responde la pregunta del proyecto. Todo lo anterior existe para que este entregable produzca datos interpretables.

Los dos experimentos

EXPERIMENTO A

Compara las dos condiciones
(con y sin detección de transiciones)

Varía: el escenario
Fijo: 4 objetos

Pregunta: ¿se reducen los
comandos falsos?

EXPERIMENTO B

Prueba cuántos objetos soporta
el método

Varía: 2, 4 y 8 objetos
Fijo: escenario 1

Pregunta: ¿se mantiene el
número de decisiones?

No se cruzan. Combinar todos los escenarios con todos los números de objetos daría 90 recorridos por persona, unas cinco horas de sesión. Inviabile por fatiga visual.

El principio es: **variar una cosa a la vez.**

Tareas

7.1 Ensayar el protocolo en simulación

```
python run_integracion.py expA
python run_integracion.py expB
python run_integracion.py protocolo
```

Léelo entero antes de traer a nadie al laboratorio. Incluye qué verificar antes, cómo calibrar, y los criterios de exclusión.

7.2 Los desvíos inducidos

Esta parte del protocolo no es opcional.

Al ensayar el sistema se descubrió lo siguiente: en un recorrido normal, el usuario hace pocas transiciones de mirada. Y si hay pocas transiciones, hay pocas oportunidades de error, y la diferencia entre las dos condiciones no llega a manifestarse en número de eventos aunque exista.

La solución: en una parte de los recorridos, el experimentador indica al sujeto que desplace la mirada en momentos marcados. Por ejemplo: mientras el robot avanza, se le pide que mire

brevemente el estímulo de giro y vuelta.

Eso cumple dos funciones:

1. **Garantiza eventos** que contar
2. **Da etiqueta exacta**: se sabe cuándo ocurrió la transición

Tipo	Proporción	Para qué
Recorridos libres	60%	Medir el desempeño en condiciones realistas
Con desvíos inducidos	40%	Garantizar eventos y tener etiqueta exacta

7.3 Preparar la sesión

Orden	Qué	Comando o criterio
1	Verificar el contrato	<code>run_integracion.py contrato</code>
2	Comprobar impedancias	<code>run_eeg.py impedancias, <5 kΩ</code>
3	Verificar el estímulo	<code>run_estimulo.py timing</code>
4	Comprobar el monitor	Debe ir a 60 Hz de verdad
5	Calibrar el clasificador	Precisión $\geq 80\%$
6	Calibrar la detección de estados	Con transiciones inducidas

Los criterios de exclusión se declaran ANTES de recoger datos, no después de verlos:

- Precisión del clasificador por debajo del 80% en calibración
- Más del 1% de fotogramas perdidos en el estímulo
- Impedancias que no bajen de 5 kΩ

7.4 Durante la sesión

- Escala de fatiga visual (1–10) cada 15 recorridos
- Cuestionario de carga de trabajo al terminar cada condición
- Descanso de 10 minutos cada 15 recorridos

Contrabalancea el orden de las condiciones entre personas. Si todos hicieran primero una condición y luego la otra, el aprendizaje del sujeto se confundiría con el efecto que se quiere medir.

7.5 Analizar

El sistema guarda cada recorrido **en cuanto termina**, no al final de la sesión. Si el programa se cae en el recorrido 30, los 29 anteriores están a salvo.

El archivo es texto legible, una línea por recorrido, con una cabecera que guarda la configuración completa. Sin eso, dentro de seis meses no sabrías con qué parámetros se corrió.

Sobre la métrica principal: se reporta la reducción **en valor absoluto**, no en porcentaje.

La razón: el porcentaje satura. Si la condición propuesta llega a cero eventos, el cociente vale 1 y ya no puede crecer, lo que haría imposible verificar si el efecto crece con la dificultad.

Checklist

Comprobación	
☐	He ensayado los dos experimentos en simulación
☐	He leído el protocolo entero
☐	Entiendo por qué hacen falta los desvíos inducidos
☐	Los criterios de exclusión están declarados por escrito
☐	El orden de condiciones está contrabalanceado
☐	Los datos se guardan tras cada recorrido
☐	He verificado que puedo releer los archivos guardados

Qué entregar

1. Los archivos de datos de todas las sesiones
2. Tablas de resultados de ambos experimentos
3. Gráficas: comparación entre condiciones, y decisiones observadas contra esperadas
4. **Dos páginas:** qué salió, qué no salió como esperabas, y qué crees que lo explica

Preguntas para pensar

1. ¿Por qué se reporta la reducción absoluta y no el porcentaje?
2. ¿Por qué el experimento B usa solo una condición y no dos?
3. Si el número de decisiones observado no coincidiera con el esperado, ¿dónde buscarías el problema?

ENTREGABLE 8

Informe integrado

Objetivo

Reunir todo lo aprendido en un documento que sirva para que otra persona pueda continuar el trabajo.

Por qué importa

Un proyecto sin documentación es un proyecto que hay que rehacer. Este entregable es el que da valor duradero a todo lo anterior.

Contenido del informe

Sec.	Título	Qué debe contener
1	Qué se hizo	Resumen de una página: qué sistema es, qué se buscaba, qué se consiguió
2	Estado del código	Qué funciona, qué falta, qué se completó. Incluye las tres partes que estaban incompletas
3	Errores encontrados	La sección más valiosa. Cada error: qué era, cómo se detectó, cómo se corrigió, qué resultado cambió
4	Montaje experimental	Fotos, medidas, calibración de la cámara, disposición de objetos y obstáculos
5	Resultados	Tablas y gráficas de los dos experimentos, con interpretación
6	Problemas abiertos	Lo que no funcionó, lo que quedó a medias, y qué habría que probar
7	Instrucciones para continuar	Qué haría falta para que otra persona retome el trabajo

Sobre la sección 3

Es la sección más valiosa del informe, aunque parezca la menos lucida.

Documentar un error que costó tres días encontrar ahorra esos tres días a la siguiente persona. Y varios de los errores de este proyecto **no daban ningún mensaje**: solo se detectaron porque alguien desconfió de un número que salía demasiado bien.

Para cada error, cuatro cosas:

1. **Qué estaba mal** — archivo, función, línea
2. **Cómo se detectó** — qué te hizo sospechar
3. **Cómo se corrigió** — el cambio concreto
4. **Qué cambió** — números antes y después

Checklist

Comprobación	
☐	Las siete secciones están escritas
☐	Cada error documentado tiene las cuatro partes
☐	Las figuras tienen pie explicativo
☐	El código entregado corre desde cero en una máquina limpia
☐	Los archivos de datos están organizados y son legibles
☐	Otra persona podría retomar el trabajo con este informe

Qué entregar

1. El informe completo en PDF
2. El código completo, con todos los cambios documentados
3. Todos los archivos de datos
4. Las figuras en alta resolución, por separado

Resumen: las diez cosas que no hay que olvidar

1. **El usuario está en reposo la mayor parte del tiempo, y eso es normal.** El comando se engancha y el robot lo mantiene. Si el robot se detuviera cada vez que el usuario deja de mirar los estímulos, el sistema sería inútil.
2. **La transición de mirada dura entre 0.14 y 0.60 segundos.** En ese intervalo la señal es una mezcla y un detector normal la confunde con una orden. Ese es el problema que ataca el proyecto.
3. **Los números salen aunque el código esté mal.** Cuatro de los ocho errores encontrados no producían ningún mensaje. Desconfía de los resultados que salen demasiado bien.
4. **Todos los parámetros están en config.py.** No pongas números sueltos en otros archivos. Si necesitas probar una variante, usa `copia_con()`.
5. **Mide el marcador impreso con una regla.** Las impresoras escalan. Un error del 5% ahí es un error del 5% en todas las distancias.
6. **Verifica las unidades del amplificador.** Si devuelve voltios en lugar de microvoltios, todo estará mal por un factor de un millón y nada fallará.
7. **Verifica el orden de los electrodos.** Conectarlos en otro orden no da error: solo da peores resultados.
8. **El estímulo se calcula con el contador de fotogramas, nunca con el reloj.** Un error de 0.1 ms por fotograma desfasa la señal 265 grados en medio minuto.
9. **La seguridad no depende de la señal cerebral.** Un obstáculo a 15 cm es peligroso independientemente de lo que el usuario esté pensando.
10. **El antiwindup del controlador es una línea fácil de olvidar.** Sin él, el robot da un acelerón inexplicable justo después de esquivar un obstáculo, y solo en esa situación.
11. **Guarda todo y documenta los errores.** Un experimento no guardado es un experimento que hay que repetir. Un error no documentado es un error que alguien volverá a cometer.

Última idea

Este código es un borrador. Tu trabajo no es solo ejecutarlo: es **entenderlo, cuestionarlo y mejorarlo**.

Los errores que encuentres y documentes son una contribución real al proyecto, no una molestia. Varios de los que ya se corrigieron se encontraron exactamente así: alguien miró un número, pensó “esto no puede ser”, y lo investigó.

*Cuando un resultado salga sospechosamente bien o sospechosamente mal,
esa es la señal para revisar el código.*